Sistemi Operativi

Francesco Lanzarini

1 Domande

1.1 Ristorante

1.1.1 Cosa è un sistema operativo?

Un sistema operativo e' un programma, piu' precisamente e' il progrmamma che viene lanciato all'accensione e terminato allo spegnimento.

1.1.2 Cosa è un programma?

Un programma e' una sequenza di istruzioni.

1.1.3 Cosa è un processo?

Un processo e' un programma che viene eseguito.

1.1.4 Come è strutturato il modello di macchina di Von Neumann?

Il modello di macchina di Von Neumann e' composto da processore che esegue le operazioni, una memoria che fornisce gli operandi delle operazioni, una memoria di massa dove memorizzare informazioni a lungo termine, bus dati per il trasferimento delle informazioni e dispositivi di esterni che gestiscono l'input/output.

1.1.5 Cosa è un processore?

Il processore e' il componente del calore che esegue le operazioni di fetch-decode-execute.

1.1.6 Cosa è un bus?

Un bus e' un canale di informazioni condiviso collegato tra piu' componenti, il suo compito e' quello di tyrasportare i dati.

1.1.7 Cosa è un controller di Input/Output?

Un controller I/O è un componente che gestisce lo scambio di dati tra un sistema di controllo e i dispositivi esterni.

1.1.8 Cosa è un device register?

Un device register è la registrazione di un dispositivo in una rete o in un sistema, associandolo a un identificativo univoco per riconoscerlo e gestirlo

1.1.9 Cosa è un device driver?

Un device driver è un software che permette al sistema operativo di comunicare con un dispositivo hardware, traducendo le sue richieste in comandi comprensibili dal dispositivo

1.1.10 Cosa può fare un processore quando è in kernel mode?

Richiedere/fornire informazioni ai bus dati

1.1.11 Cosa può fare un processore quando è in user mode?

Eseguire le istruzioni dei programmi che non siano SO

1.1.12 Come e in che casi avviene il passaggio da user mode in kernel mode?

Il passaggio avviene principalmente in tre casi: system call, interrupt opppure trap/exceptions

1.1.13 Cosa è una system call?

La system call e' l'interfaccia con cui un programma in User Mode chiede al sistema operativo un servizio kernel.

1.1.14 Cosa è un Process Control Block?

Un PCB e' una struttura dati che serve al sistema operativo per la memorizzazione delle informazioni dei processi.

1.1.15 Cosa vuole dire scheduling?

Per scheduling si intende l'operazione del sistema operativo atta a gestire l'esecuzione da parte della CPU di processi "ready".

1.1.16 Come vengono sospesi i processi in attesa di operazioni di I/O?

I processi vengono sospesi grazie ad una system call.

1.1.17 Cosa significa multitasking?

Per multitasking si intende la capacita' della CP di eseguire piu' processi contemporaneamente.

1.1.18 Cosa è un interrupt e quando avviene?

Un interrupt (interruzione) è un segnale inviato al processore per interrompere il flusso di esecuzione corrente e richiedere l'esecuzione immediata di una speciale routine del kernel, puo' avvenire per diversi motivi tipo la digitazione di un carattere.

1.1.19 Una system call o un interrupt fa passare il controllo dal processo in esecuzione al sistema operativo. Cosa deve fare il processore per consentire di poter riprendere successivamente l'esecuzione del processo dal punto dell'interruzione?

Per consentire la corretta proseguita del processo il sistema operativo ha a disposizione tutti i dati necessari nella PCB del singolo processo.

1.2 Concorrenza

1.2.1 Quale è la differenza fra programma e processo?

Il programma e' statico: e' un testo che contiene una sequenza di istruzioni. Un processo invece e' dinamico: e' l'esecuzione delle istruzioni che sono contenute nel programma.

1.2.2 Cos'è l'assioma di finite progress? perché è necessario?

L'assioma di finite progress e' l'assioma che ci assicura che ogni processo viene eseguito ad una velocita' finita, ignota ma non nulla. Esso e' importante perche' introduce il problema maggiore della concorrenza: non essere in grado di determinare numericamente quanto un processo impiegera' ad essere eseguito costringendoci a un diveso approccio per la sincronizzazione dei programmi.

1.2.3 Quali sono gli stati dei processi?

Gli stati dei processi sono 3:

1. Ready: il processo e' pronto ad essere eseguito m,a il processore e' impegnato in altre attivita'.
2. Waiting: il processo sta attendendo un qualche evento esterno, per esempio se deve ricevere un informazione.
3. Running: il processo sta venendo eseguito.

1.2.4 Cos'è la concorrenza?

Per concorrenza si intende la notazione e le tecniche per risolvere problemi, quali comunicazione e sincronizzazioni, associati all'esecuzione concorrente (ovvero simultanea).

1.2.5 Qual è la differenza fra concorrenza reale e apparente?

Per concorrenza reale si intende la capacita' delle infrastrutture di calcolo distribuite oppure di processore multicore di eseguire piu' processi contemporaneamente. Invece la concorrenza apparente e' un illusione data dalla velocita' con cui i processori riescono ad alternare l'esecuzione di processi diversi.

1.2.6 I problemi da risolvere per la programmazione concorrente sono diversi fra parallelismo reale o apparente?

No, i problemi di comunicazione e sincronizzazione sono sempre gli stessi percheanche se in un sistema multiprocessore i processi sono intervallati nel tempo (interleaving) mentre in un sistema multicore sono eseguiti simultaneamente ma sono alternati nello spazio di memoria (overlapping).

1.2.7 Cosa è una race condition?

Una race condition e' una condizione per la quale il risultato finale dell'esecuzione dipende dalla temporizzazione con cui vengono eseguiti i processi.

1.2.8 Cosa è una proprietà di un programma?

La proprieta' di un programma e' un attributo dello stesso che rimane vero per ogni istanza di esecuizione dello stesso.

1.2.9 Quali sono le proprietà di safety?

Le proprieta' di safety e' la proprieta' che garantisce che il programma vada nella direzione voluta. Esse sono:

- Mutua esclusione: in ogni istante un solo processo puo' accedere alla singola risorsa.
- Assenza di deadlock.

1.2.10 Quali sono le proprietà di liveness?

La proprieta' di liveness e' la proprietà che ci garantisce che il programma avanza e non si ferma. Esse sono:

- Assenza di starvation.

1.2.11 Cosa è il deadlock?

Il deadlock e' un fenomeno per cui 2 o piu' processi hanno bisogno della mutua esclusione sulle stesse risorse per cui si bloccano visto che non se le riescono a liberare a vicenda.

1.2.12 Cosa è la starvation?

La starvation e' un fenomeno per cui un processo attende in modo infinito l'accesso ad una risorsa.

1.2.13 Cosa è una azione atomica? (quali operazioi assumiamo atomiche nella nostra trattazione?)

Un'azione atomica e' un'istruzione che sicuramente il processore esegue in un ciclo di clock, per cui siamo sicuri che quando inzia ad essere esegiuta essa sara' anche terminata prima che il processore ne esegua un'altra. Nella nostra trattazione consideriamo atomiche le apoerazioni di assegnamento alle variabili di valori costanti.

1.2.14 Cosa è una sezione critica?

Una sezione critica, o critical section, e' una parte di programma che che utilizza una o piu' risorse condivise. Il nostro obiettivo e' che queste sezioni critiche siano eseguite in modo mutualmente esclusivo.

1.2.15 Perché il compilatore non può individuare autonomamente le sezioni critiche?

Il compilatore non e' in grado di individuare le sezioni critiche perche' esso non capisce la logica del programma, compila e basta.

1.2.16 Come possiamo definire le sezioni critiche tramite le proprietà?

Le sezioni critiche possono essere definite tramite le seguenti proprietà:

1. Mutua esclusione: solo un processo alla volta deve star eseguendo una sezione critica tra tutte quelle che accedono alla stessa risorsa condivisa.
2. Assenza di deadlock: non e' ammissibile uno scenario in cui tutti i processi vengono bloccati.
3. Assenza di delay non necessari: un processo che non sta eseguendo una sezione critica non deve ritardare inutilmente l'esecuzione della sezione critica di un altro.
4. Asenza di starvation (Eventual Entring): prima o poi ogni processo inizia ad eseguire la propria sezione critica.

1.2.17 Come funziona l'algoritmo di Dekker?

```
shared int turno = P;
shared bool needp = false;    shared bool needq = false;

process P {                                    process Q {
    /* entry protocol */                           /* entry protocol */
    needp = true;                                  needq = true;
    while(needq){                                  while(needp){
        if(turn == Q){                                 if (turn == P){
            needp = false;                                 needq = false;
            while (turn == Q)                              while(turn == P)
                ; /* do nothing */                            ; /* do nothing */
            needp = true;                                  needq = true;
        }                                              }
    }                                              }
    critical section                               critical section
    needp = false; turn = Q;                       needq = false; turn = P;
    non-critical section                           non-critical section
}                                              }
```

Come si puo' vedere dal codice ci sono due variabili booleane condivise che indicano quando i processi vogliono entrare in mutua esclusione, con il primo while invece si va a controllare se l'altro processo vuole la mutia esclusione: nel caso non la voglia non si controlla neanche il turno e il processo entra in critical section. Alla fine della critical section il processo passa il turno all'altro.

1.2.18 Algoritmo di Peterson per due processi, come funziona?

```
shared bool needp = false; shared bool needq = false;
shared int turn;


process P {                                    process Q{
    while(true){                                   while (true){
        /* entry protocol */                           /* entry protocol */
        needp = true;                                  needq = true;
        turn = Q;                                      trurn = P
        while (needq && turn != P)                     while (needp && turn != Q)
            ; /* do nothing */                            ; /* do nothing */
        critical section                               critical section
        needp = false;                                 needq = false;
        non-critical section                           non-critical section
    }                                              }
}                                              }
```

In questo algoritmo il turno viene passato prima dell'entrata nella critical section

1.2.19 Generalizzazzione dell Algoritmo di Peterson per N processi, come funziona?

```
shared int [] stage = new int [N]; /* 0-initialized */
shared int [] last = new int[N];  /* 0-initialized */

process P_i {  /* i = 0, ..., N-1 */
    while (true){
        /* entry protocol */
        for (int j = 0; j < N; j++){
            stage[i] = j;
            last[j] = i;

            for (int k = 0; k < N; k++) {
                if (i != k) {
                    while (stage[k] >= stage[i] && last[j] == i)
                        ; /* do nothing */
                }
            }
        }
        critical section
        stage[i] = 0;
```

```
            non-critical section
        }
    }
```

1.2.20 Sono possibili soluzioni hardware per il problema delle sezioni critiche? Quali?

Si, ci sono possibili soluzioni hardware: si potrebbe pensare di modificare il set di istruzioni hardware per semplificare la realizzazione di sezioni critiche coem il mascheramento degli interrup. Questa tecnica pero' non va bene perche' e' troppo rischioso delegare ai singoli programmi la responsabilita' di riattivare gli interrupt, riduce il grado di parallelismo e non funziona su sistemi multicore. Altro modo sono le istruzioni speciali Test & set ovvero realizzare in modo atomico due istruzioni Ts(x,y) := <y = x, x = 1>.

```
shared int lock = 0;

process P{
    int vp;
    while(true) {
        do{
            TS(lock, vp);
        } while (vp);
        critical section
        lock = 0;
        non-critical section
    }
}
```

```
process Q{
    int vq;
    while(true){
        do{
            TS(lock, vq);
        } while(vp);
        critical section
        lock = 0;
        non-critical section
    }
}
```

Entra in critical section il primo dei due processi che riesce a "fotografare" lock a 0 e a settarlo a 1 per l'altro. La starvation e' evitata con qualche assunzione in piu'.

1.2.21 In quali casi può applicare la disabilitazione degli interrupt?

La disabilitazione degli interrupt si puo' applicare solo sui sistemi a singolo processore.

1.2.22 SpinLock: cosa sono? quale supporto hardware serve per utilizzarli?

Gli SpinLock sono delle quasi sezioni critiche che sono realizzate con istruzioni speciali e serve che l'hardware supporti la loro esecuzione in modo atomico.

1.2.23 Gli spinlock forniscono una soluzione al problema delle sezioni critiche? (tutte le quattro proprietà sono soddisfatte?)

Si, ma l'assenza di starvtion ha bisogno di ulteriori assunzioni per essere garantita.

1.2.24 Semafori: cosa sono? Quali operazioni supportano?

Il semaforo è un tipo di dato astratto per il quale sono definite due operazioni:

- V: essa viene invocata per inviare un segnale, in pratica fa finire la critical section
- P: serve per attendere un segnale, in pratica serve per entrare in una critical section.

Queste sono due azioni atomiche. Inoltre un semaforo puo' essere visto come una variabile intera che non deve essere mai negativa: l'operazione P aspetta che il semaforo sia positivo e decrementa di 1 mentre l'operazione V incrementa di 1 il semaforo.

1.2.25 Qual è l'invarinte del semaforo?

L'invariante è: $n_p \leq n_v - init$ con n_p il numero di operazioni P completate, n_v il numero di operazioni V completare e $init$ il valore inixiale del semaforo.

1.2.26 Cosa è il valore del semaforo?

Il valore del semaforo è $n_v - init - n_p$ e non deve mai essere minore di 0.

1.2.27 Come si implementa una critical section tramite i semafori?

Una critical section con semafori si implementa iniziando con un'operazione P per entrare nella critical section e per terminarla si fa un'operazione V

1.2.28 Cosa significa che un semaforo è fair?

Un semaforo viene definito fair quando rispetta una politica FIFO ovvero che il primo processo che ha richiesto di entrare in critical section doveva essere quello che ci entrava per primo tra tutti quelli in attesa. In altre parole garantisce l'assenza di starvation.

1.2.29 Come possono essere implementati i semafori in un sistema operativo?

```
void P() {
    if(value > 0)
        Value--;
    else {
        pid = <Process id>;
        queue.add(pid);
        suspend(pid);
    }
}
```

```
void V(){
    if(queue.empty())
        Value++;
    else {
        pid = queue.remove();
        wakeup();
    }
}
```

Con suspend il sistema operativo mettere il processo nello stato di waiting mentre con wakeup lo mette in stato di ready.

1.2.30 Perché conviene implementare le critical section nei processi utente con i semafori anche se l'implementazione dei semafori usa busy-wait (spinlock o Peterson)?

Perche' queste operazioni per funzionare bene partono dall'assunto che esse siano eseguite in modo atomico per cui in un sistema uniprocessore si possono disattivare gli interrupt mentre in un multicore bisogna garantire la mutua esclusione, inoltre il busy-waiting e' comunque ridotto al minimo infatti le sezioni critiche di P e Q sono molto brevi.

1.2.31 Semafori binari, cosa sono?

I semafori binari sono particolari tipi di semafori in cui il valore puo' assumere solo valori 0 o 1.

Cosa è il potere espressivo di un paradigma di programmazione concorrente? Semafori e semafori binari hanno lo stesso potere espressivo? perché?